

第5次实验 CPU设计 (I)

1 实验目的

- (1) 掌握单总线结构MIPS处理器（硬布线控制器、微程序控制器）、单周期MIPS处理器（硬布线控制器）、单周期RISC-V处理器（硬布线控制器）设计的基本原理。
- (2) 对已有的5条指令单总线结构MIPS处理器（微程序控制器）进行修改，增加1条add指令（共6条指令）。
- (3) 鼓励同学们对已有的5条指令单总线结构MIPS处理器（硬布线控制器）进行修改，增加1条add指令（共6条指令）。

2 实验环境

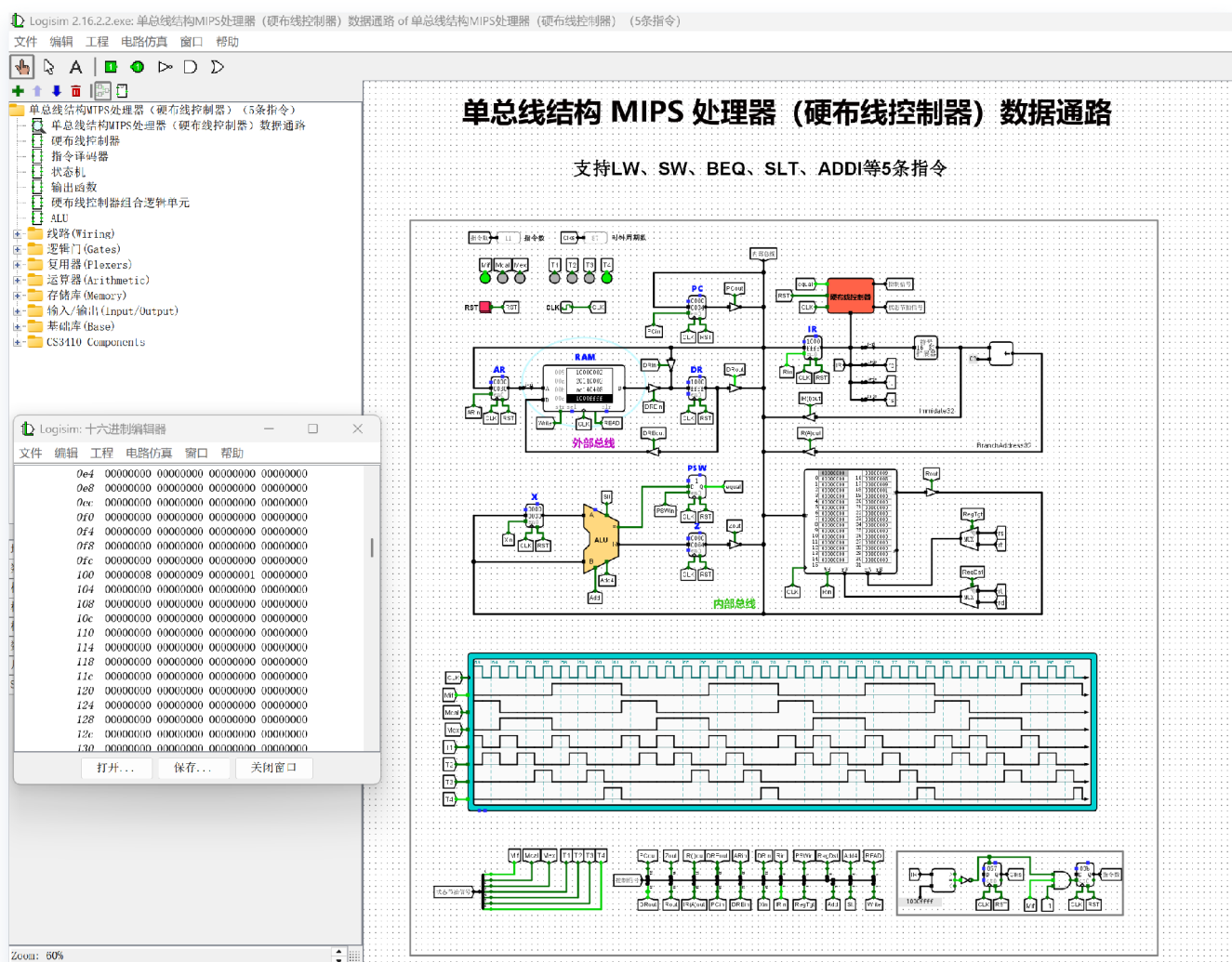
- (1) Windows系统下运行Logisim软件（需安装JDK）。
- (2) MARS 4.5、RARS 1.5汇编工具。

3.1 验证性实验

(1) 单总线结构 MIPS 处理器（硬布线控制器）（5条指令）

test1

截图

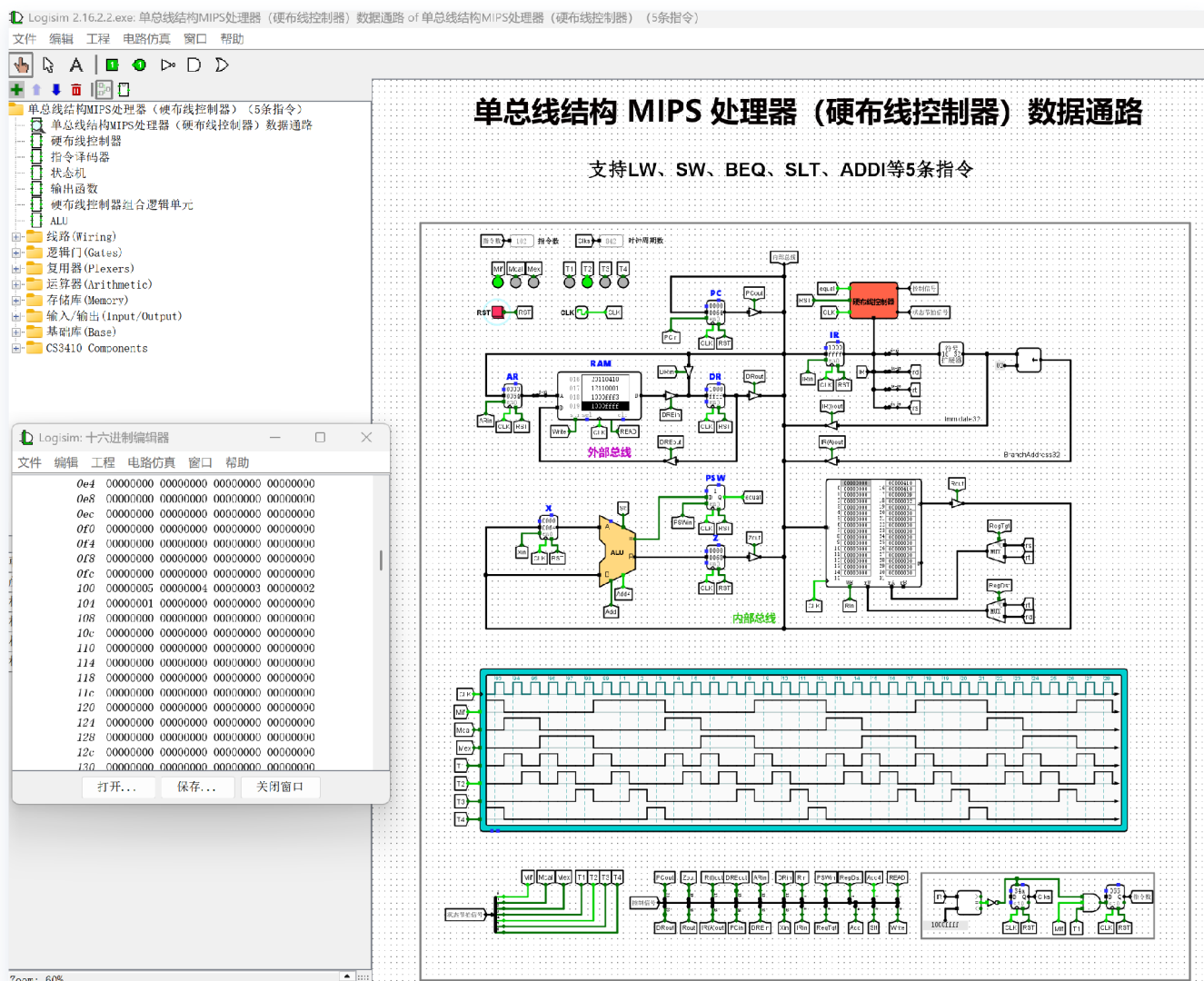


实验分析

运行 test1.hex 后，程序最终停在结束死循环。截图中 RAM 的 100、101、102 单元分别为 00000008、00000009、00000001，与 test1.asm 中的预期结果一致。其中 102 单元的 00000001 来自 slt 指令对 8 和 9 的比较结果，说明该 5 条指令单总线 MIPS 处理器能够正确完成立即数写寄存器、访存、比较以及分支控制。程序没有继续执行 loop1 中的两条指令，也与控制流设计相符。

sort3_mips_bus

截图

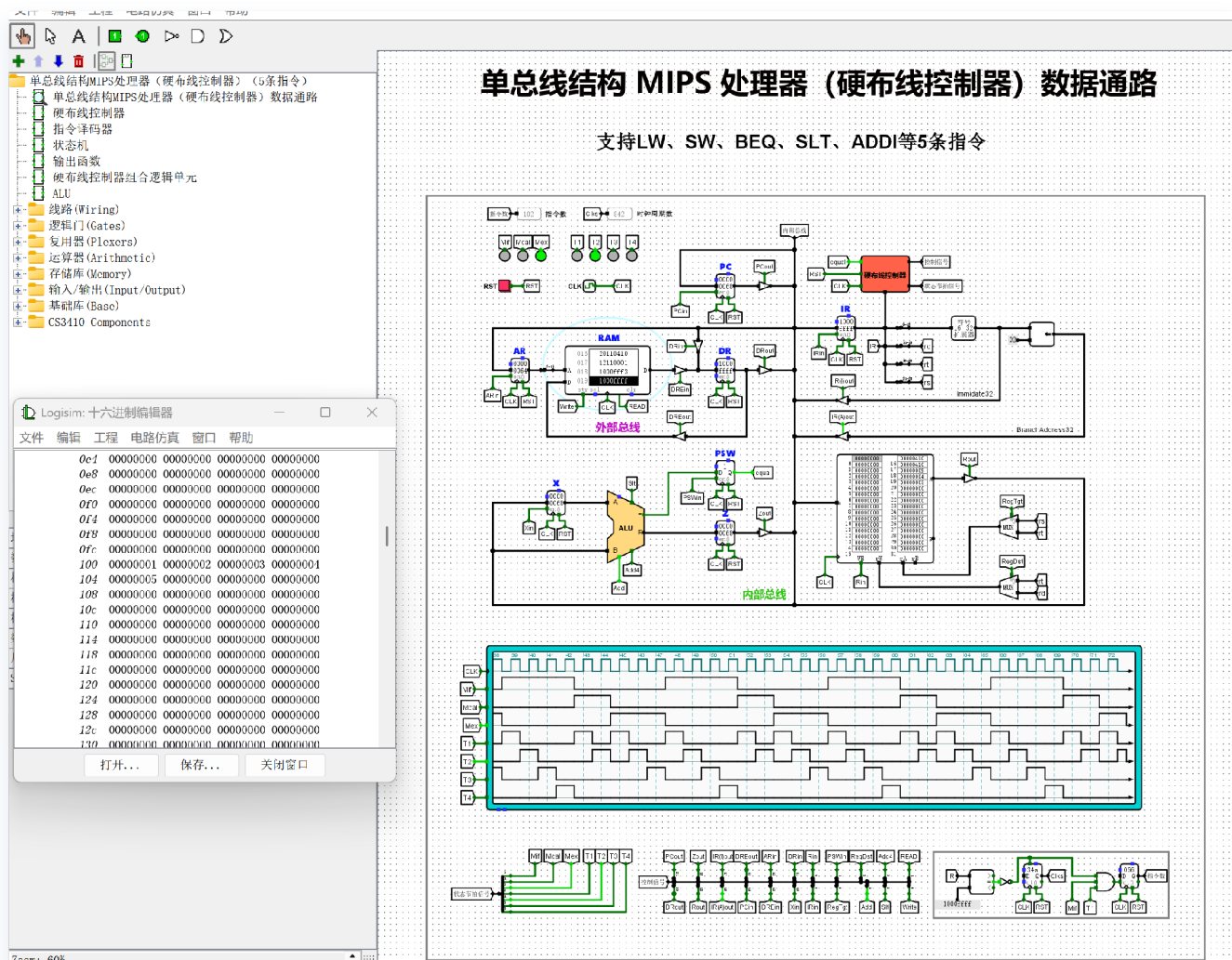


实验分析

运行 `sort3_mips_bus.hex` 后，程序最终进入结束死循环。截图中 RAM 的 100~104 单元依次为 00000005、00000004、00000003、00000002、00000001，说明原始数据 3、1、5、2、4 已按从大到小排好序。排序结果正确，处理器能够正确配合 `lw`、`sw`、`slt`、`addi` 和 `beq` 完成多次比较、交换和循环控制。整个过程中未出现异常，单总线数据通路在多轮访存和分支情况下工作正常。

sort4_mips_bus

截图



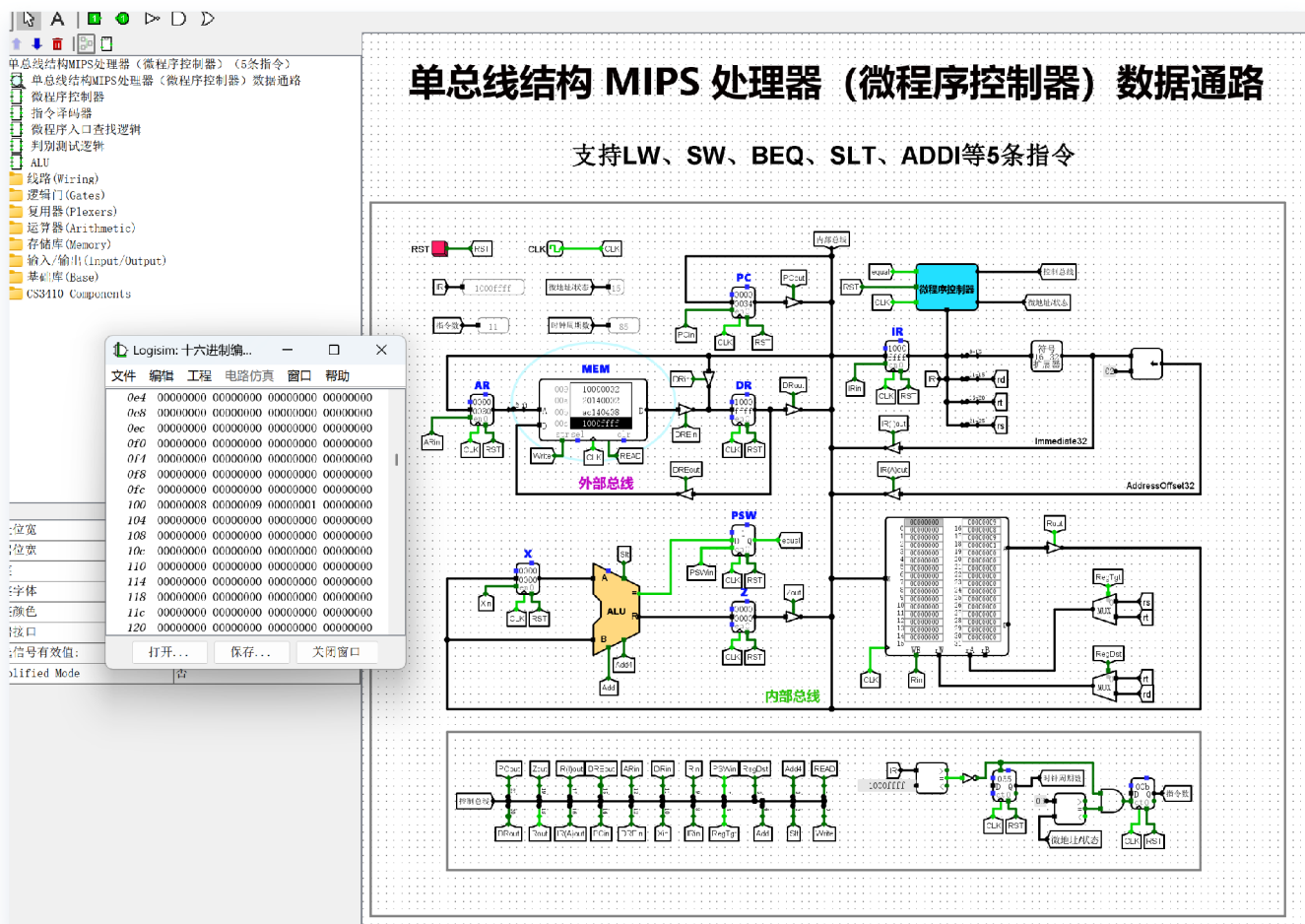
实验分析

运行 `sort4_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 100~104 单元依次为 00000001、00000002、00000003、00000004、00000005，说明待排序数据已经按从小到大的顺序写回存储器。排序结果正确，该处理器不仅能完成降序排序，也能通过改变比较方向实现升序排序。这也反映出 5 条指令版本的数据通路和硬布线控制器能够正常运行排序程序。

(2) 单总线结构 MIPS 处理器（微程序控制器）（5条指令）

test1

截图



实验分析

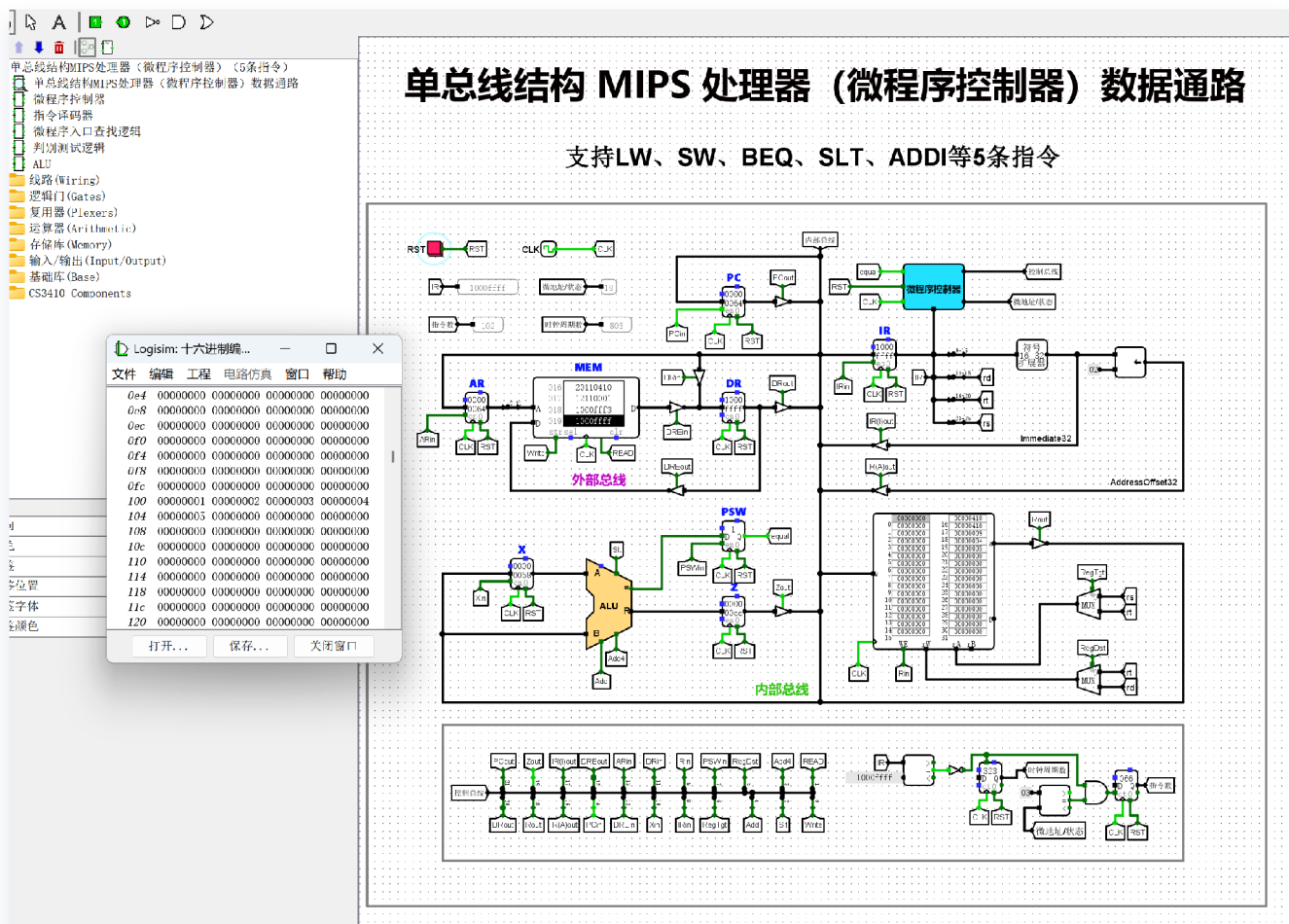
在微程序控制器版本中运行 `test1.hex` 后，程序最终进入结束死循环。截图显示 RAM 的 100、101、102 单元分别为 00000008、00000009、00000001，与汇编程序预期完全一致。102 单元保存的是 `slt` 比较 8 和 9 后得到的结果，说明微程序控制器对 `lw`、`sw`、`addi`、`slt`、`beq` 五条指令的微操作控制正确。微程序入口查找和后续微地址转移过程也没有出现异常。

sort3_mips_bus

截图

支持LW、SW、BEQ、SLT、ADDI等5条指令





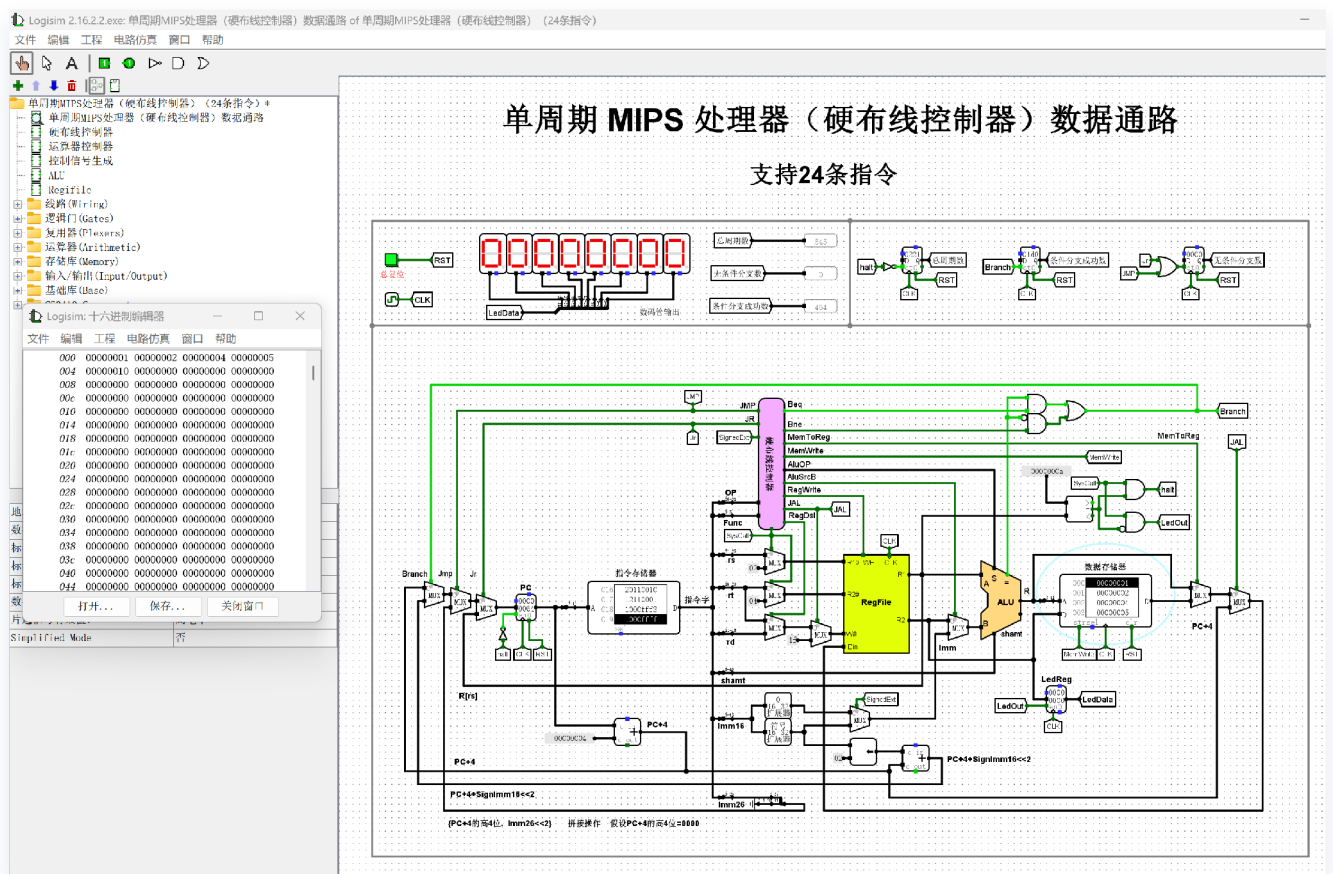
实验分析

运行 `sort4_mips_bus.hex` 后，程序最终进入结束死循环。截图中 RAM 的 100~104 单元依次为 00000001、00000002、00000003、00000004、00000005。排序结果实现了从小到大排列，与预期结果一致。微程序控制器不仅能支持基本访存与比较，还能在多轮条件判断下正确更新寄存器和存储器内容。微程序控制方式同样能够正常运行这类排序程序。

(3) 单周期 MIPS 处理器（硬布线控制器）(24条指令)

sum_mips

截图



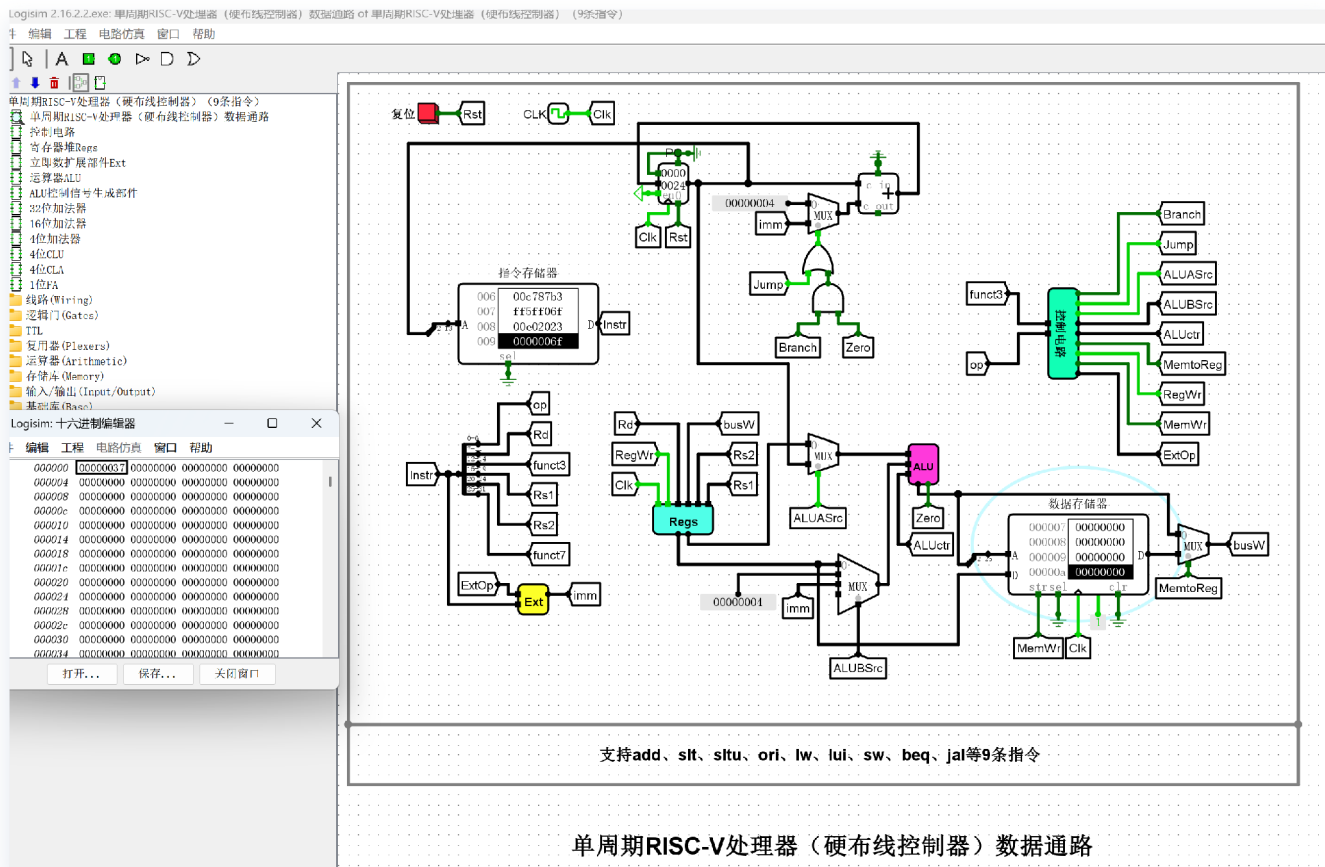
实验分析

运行 `sort4_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 `000~004` 单元依次为 `00000001`、`00000002`、`00000004`、`00000005`、`00000010`。这表明原始数据已经被正确地按升序写回存储器，数据已正确完成排序。程序能够在单周期结构下完成全部循环和条件分支，控制信号生成电路对不同指令类型的译码也是正确的。整个运行过程中未出现异常结果。

(4) 单周期 RISC-V 处理器（硬布线控制器）（9条指令）

sum_riscv

截图

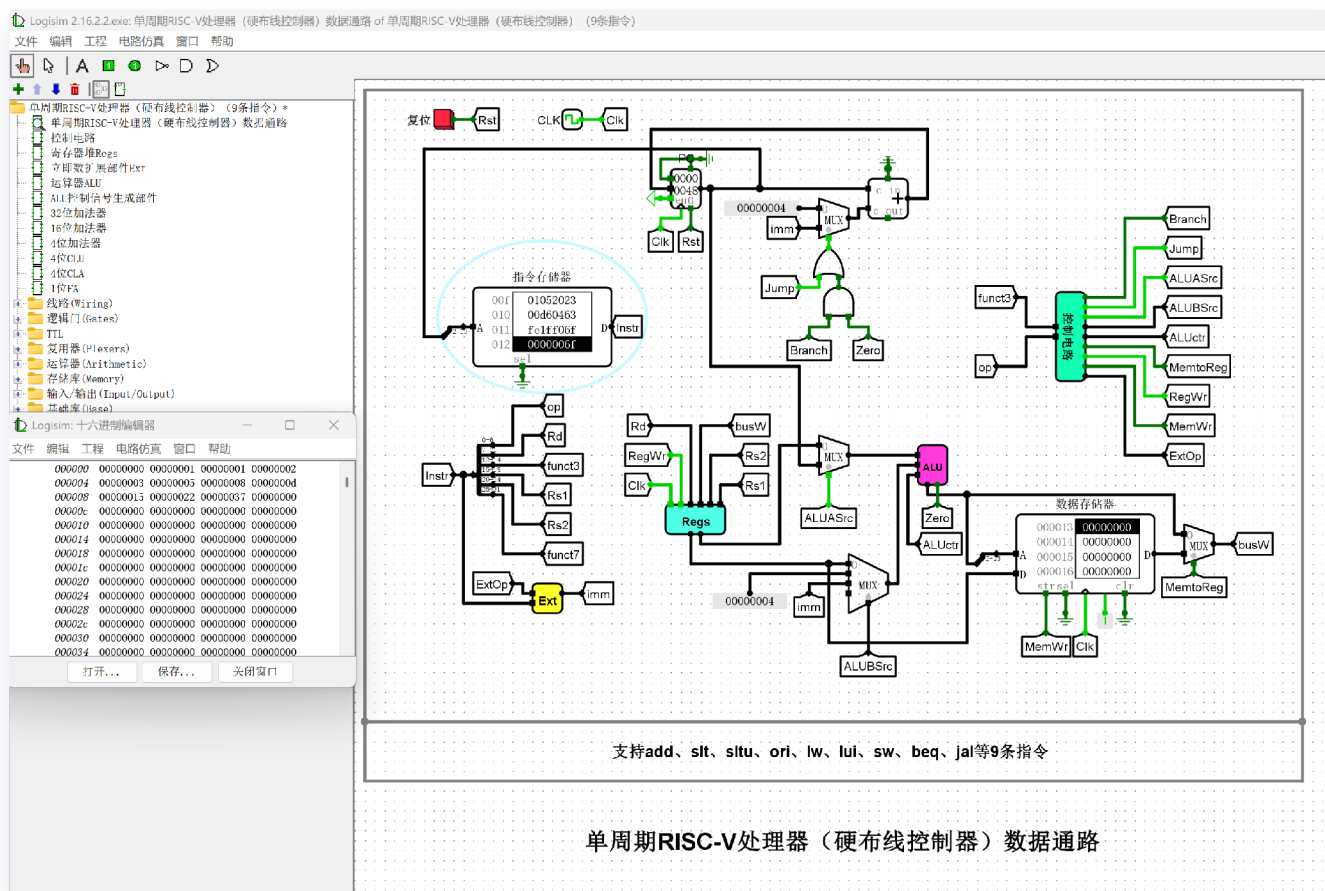


实验分析

从截图可见程序最终停在结束死循环，存储器结果与预期一致。结合 `sum_riscv.asm` 可知，该程序应将 `1+2+...+10` 的结果写入数据存储器，实验现象说明单周期 RISC-V 处理器能够正确执行 `ori`、`add` 指令、`beq`、`jal` 和 `sw` 等指令。虽然截图中未完整显示结果单元的具体数值，但程序已经到达结束死循环，累加循环和写回流程没有出现异常。RISC-V 数据通路和硬布线控制器在这一程序中运行正常。

fib_riscv

截图

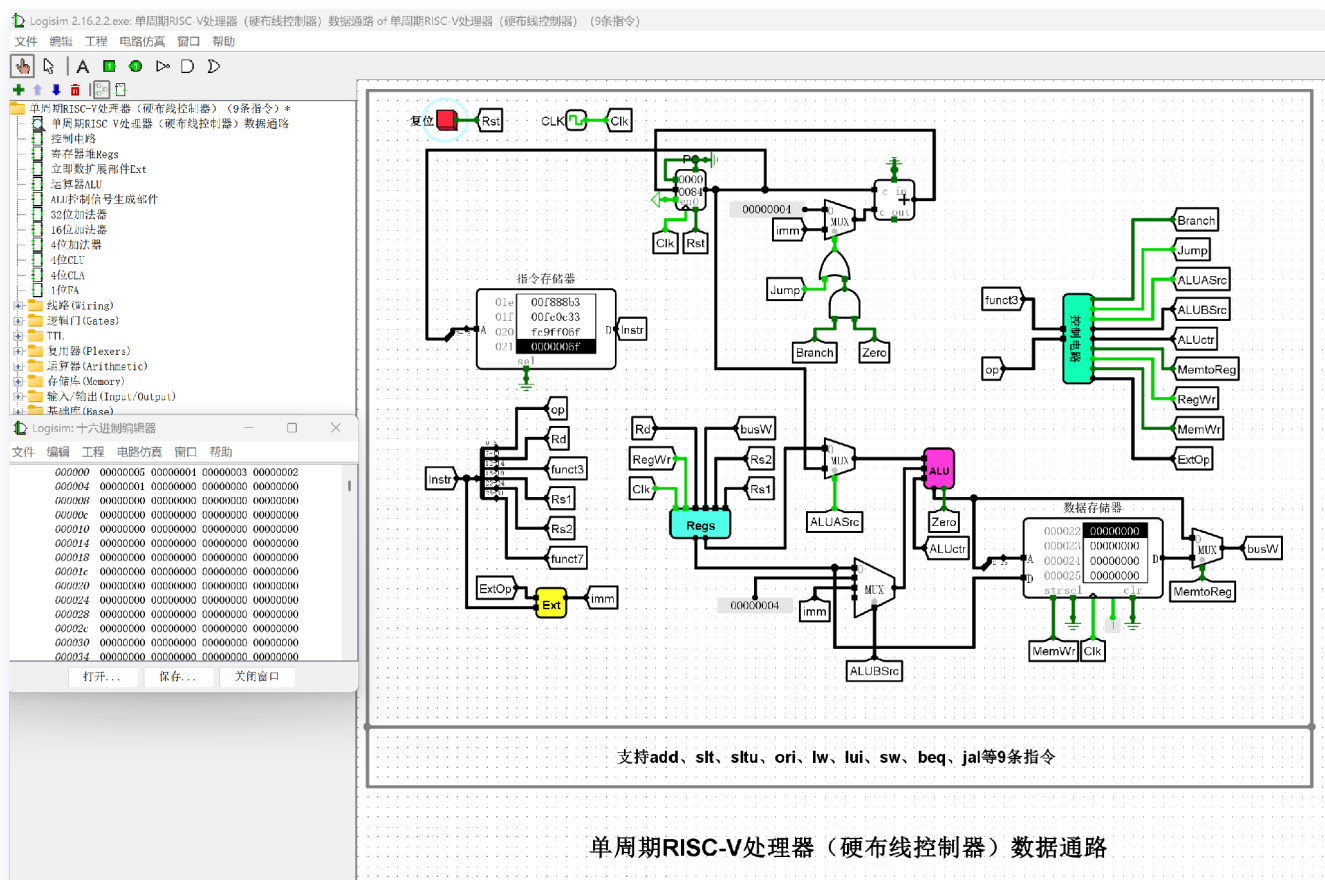


实验分析

从截图可见程序最终停在结束死循环，存储器结果与预期一致。结合程序内容可知， `fib_riscv` 会把 Fibonacci 数列前若干项依次写入数据存储器，因此可以判断处理器能够正确完成递推加法、地址更新和循环分支。该程序比简单累加更依赖寄存器之间的数据传递，能够运行成功说明 `add` 指令、`ori`、`sw`、`beq` 和 `jal` 等指令配合正常。9 条指令版本的单周期 RISC-V 处理器已经具备运行数列程序的能力。

sort3_riscv

截图

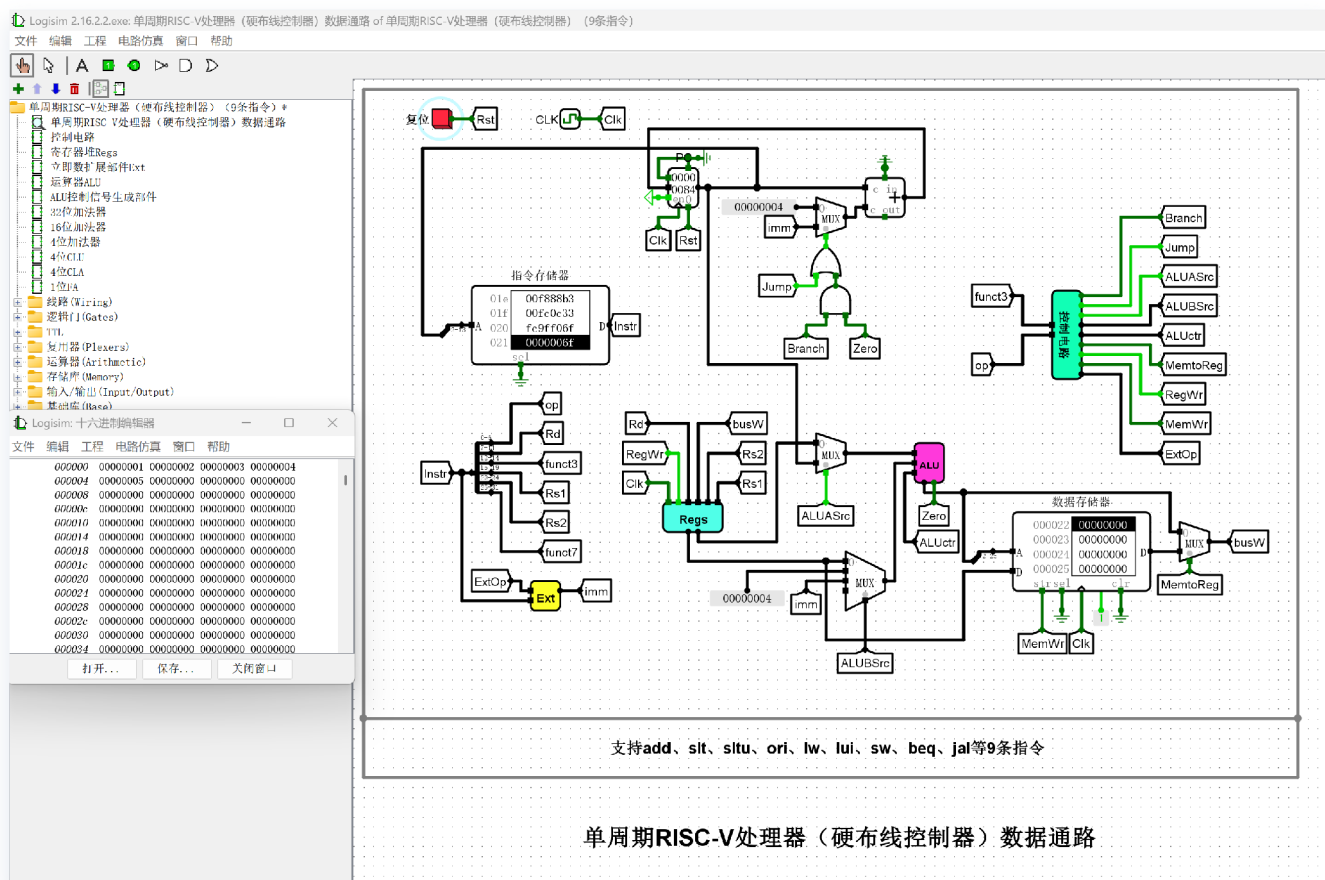


实验分析

从截图可见程序最终停在结束死循环，存储器结果与预期一致。结合 `sort3_riscv.asm` 可知，该程序会把原始数据排序为从大到小的顺序，因此可以判断处理器能够完成多次无符号比较、交换、访存和循环跳转。排序程序对控制路径要求较高，能够运行成功表明 `sltu`、`lw`、`sw`、`beq`、`jal` 等指令均已正确实现。该实验验证了单周期 RISC-V 处理器执行复杂控制程序的能力。

sort4_ricv

截图



实验分析

从截图可见程序最终停在结束死循环，存储器结果与预期一致。根据 `sort4_riscv.asm`，程序目标是将数据按从小到大排序写回存储器，因此可以判断升序排序过程已经正确完成。该程序同样涉及多层循环、比较和交换，数据通路与控制电路能够正常支持这类排序程序，运行结果符合预期。

回答问题

(1) 针对test1.asm程序段

```

loop1:
    addi $s4,$zero,2      #s4=2
    sw $s4,1032($zero)    #s4=2 保存到(1032, 即102单元)
        
```

这条指令不会执行到
这条指令不会执行到

为何上述两条指令不会执行到？

因为前面的 `beq $zero,$zero,loop2` 会无条件跳转到 `loop2`，随后 PC 会一直在 `loop2` 处循环，不再按顺序执行后面的两条指令。截图中的运行结果也印证了这一点：RAM 的 102 单元最终写入的是前面 `sw $s3,1032($zero)` 的结果 `00000001`，而不是 `loop1` 中准备写入的数值。也就是说，程序控制流在进入 `loop2` 后已经被固定住，后续指令地址虽然仍在存储器中，但实际上不会

被取到和执行。

(2) 为何安排死循环？若不采用死循环，针对此种情况，MIPS架构还可以采用什么方式？ (课外扩展)

实验电路中没有操作系统，也没有专门的 `halt` 指令，因此程序如果“自然结束”，PC 还会继续向后取指，可能读到未定义的存储单元内容，造成不可预期的执行结果。安排死循环的作用，就是让处理器停留在一个确定的、可观察的结束状态，便于查看 RAM、寄存器和周期数。若不采用死循环，在完整的 MIPS 体系中还可以通过 `syscall`、`break`、异常或陷入处理等方式结束程序，由系统接管后续流程。

(3) 在该单总线结构 MIPS 处理器（硬布线控制器）的数据通路上，能够运行 求累加和程序、计算费波那契数列程序吗？为什么？

不能完整运行。因为这两个程序都需要寄存器之间的加法运算，例如累加和程序中的 `add $s3,$s3,$s1`，以及 Fibonacci 程序中的 `add $s6,$s4,$s5`。而原 5 条指令版本只支持 `lw`、`sw`、`beq`、`slt`、`addi`，其中 `addi` 只能完成寄存器与立即数相加，不能代替 `add` 指令完成两个寄存器操作数的加法。因此在未扩展 `add` 指令之前，这两类程序无法在该数据通路上正确执行。

(4) 针对单总线结构MIPS处理器（硬布线控制器）的下述电路进行分析。

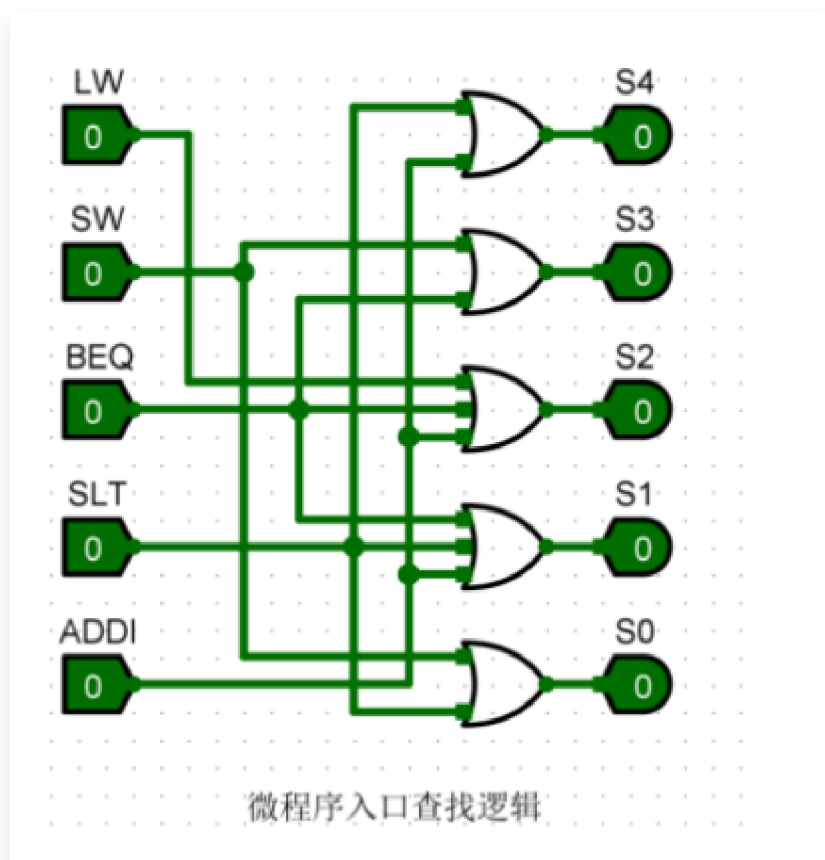
a)硬布线控制器

b)状态机

硬布线控制器部分根据指令译码结果、时钟以及比较结果 `equal` 直接组合产生各条控制信号，不需要控制存储器，因此响应速度较快。图中的状态机则负责描述处理器在取指、译码、执行、访存、写回等阶段之间的转移关系，不同指令会在状态机中走不同路径。对于 `lw`、`sw`、`beq`、`slt`、`addi` 等指令，控制器会结合当前状态输出 `PCout`、`ARin`、`DRin`、`Rin`、`Write` 等控制信号，使数据沿单总线按既定节拍流动。也就是说，状态机决定“当前处于哪一步”，而硬布线组合逻辑决定“这一步该发哪些控制信号”。

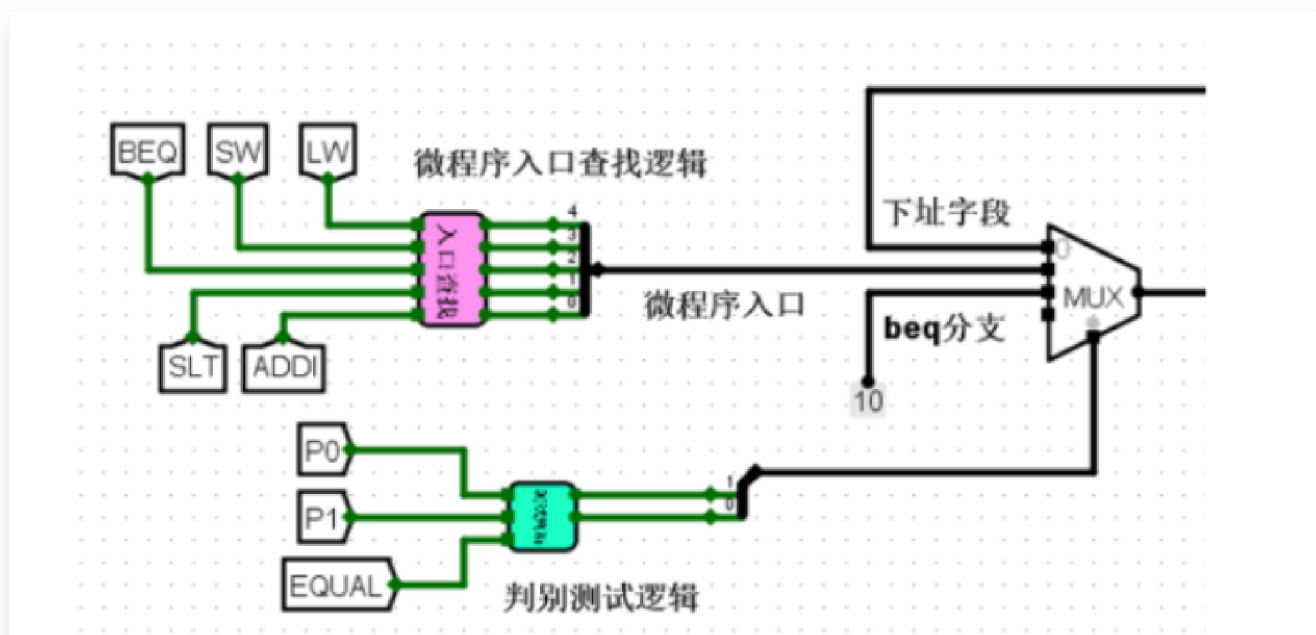
(5) 分析单总线结构MIPS处理器（微程序控制器）的下述电路原理

a)微程序入口查找逻辑



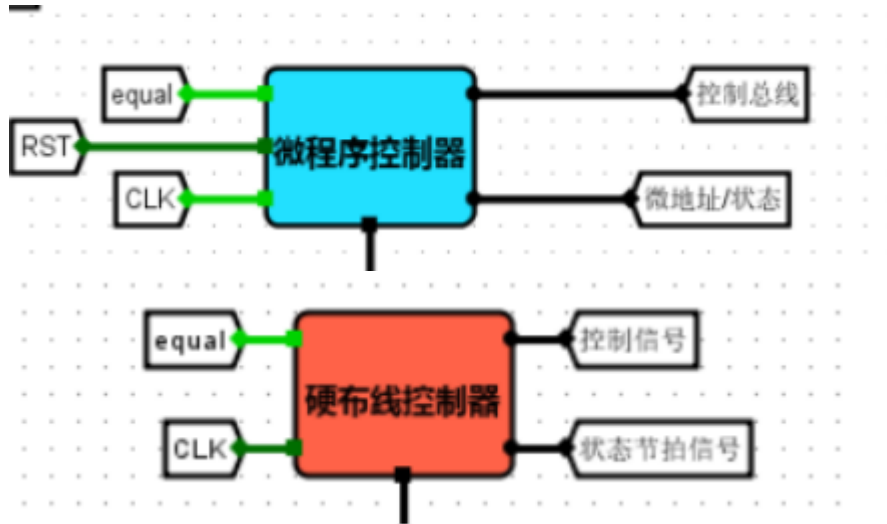
b)判别测试逻辑

微程序入口查找逻辑的作用，是根据当前指令的译码结果为不同指令选出相应的微程序入口地址。这样在公共取指结束后，`lw`、`sw`、`beq`、`slt`、`addi` 会分别跳转到各自的微程序段继续执行。判别测试逻辑则负责根据微指令中的判别字段以及外部条件信号决定下一条微指令地址，例如在 `beq` 执行时利用 `equal` 信号选择是否转向分支相关的后续微操作。两部分配合后，微程序控制器就能够像“按流程表执行”一样完成不同指令的控制。



(6) 结合工作原理，比较下述微程序控制器和硬布线控制器引脚上的不同

微程序控制器的引脚除了时钟、复位和条件输入外，还需要与“微地址/状态”相关的接口，用来给出当前微指令地址并控制下一微地址的选择，因此它体现的是“由控制存储器读出控制字”的工作方式。硬布线控制器则更多体现为状态信号、译码信号和条件信号输入，再直接输出控制总线与状态节拍信号，本质上是组合逻辑加状态机驱动。前者便于修改和扩展指令，只需调整微程序或入口逻辑；后者不依赖控制存储器，结构更直接，速度通常更快，但新增指令时需要重新设计状态与逻辑。两者引脚差异正反映了两种控制方式在实现机制上的不同。



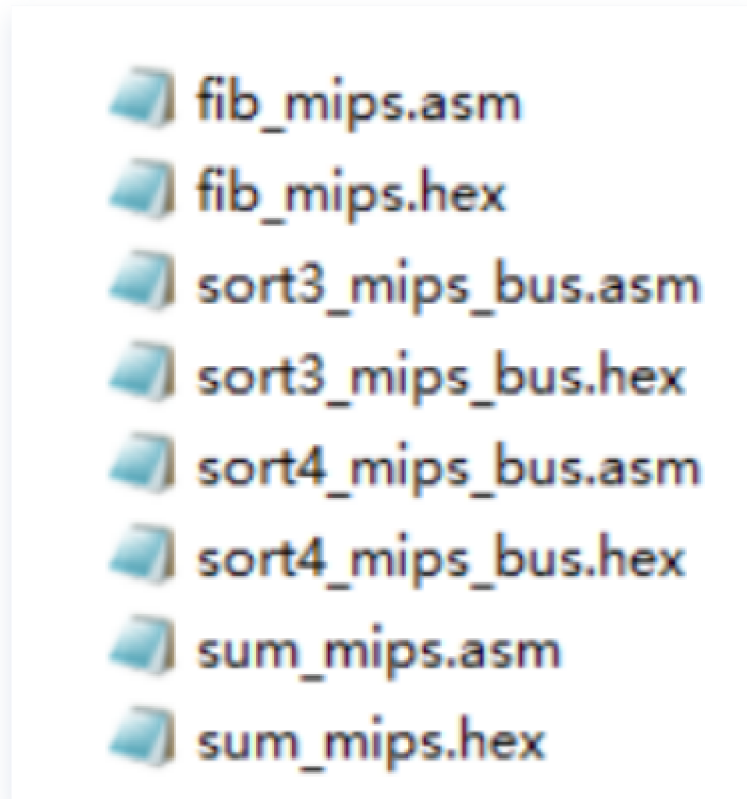
(7) 请在单总线结构 MIPS 处理器（微程序控制器）的数据通路上运行程序：

test1.hex、sort3_mips_bus.hex、sort4_mips_bus.hex

观看RAM存储器中的结果是否正确？

结果是正确的。结合前面的三张实验截图可以看到，`test1.hex` 运行后 RAM 的 100、101、102 单元分别为 00000008、00000009、00000001；`sort3_mips_bus.hex` 运行后 RAM 的 100~104 单元变为 00000005、00000004、00000003、00000002、00000001；`sort4_mips_bus.hex` 运行后则变为 00000001、00000002、00000003、00000004、00000005。这些结果都与各程序预期一致，微程序控制器能够完成写数、比较、交换和循环控制。

(8) 针对单周期MIPS处理器，测试程序，观察结果（拷屏+文字解释）

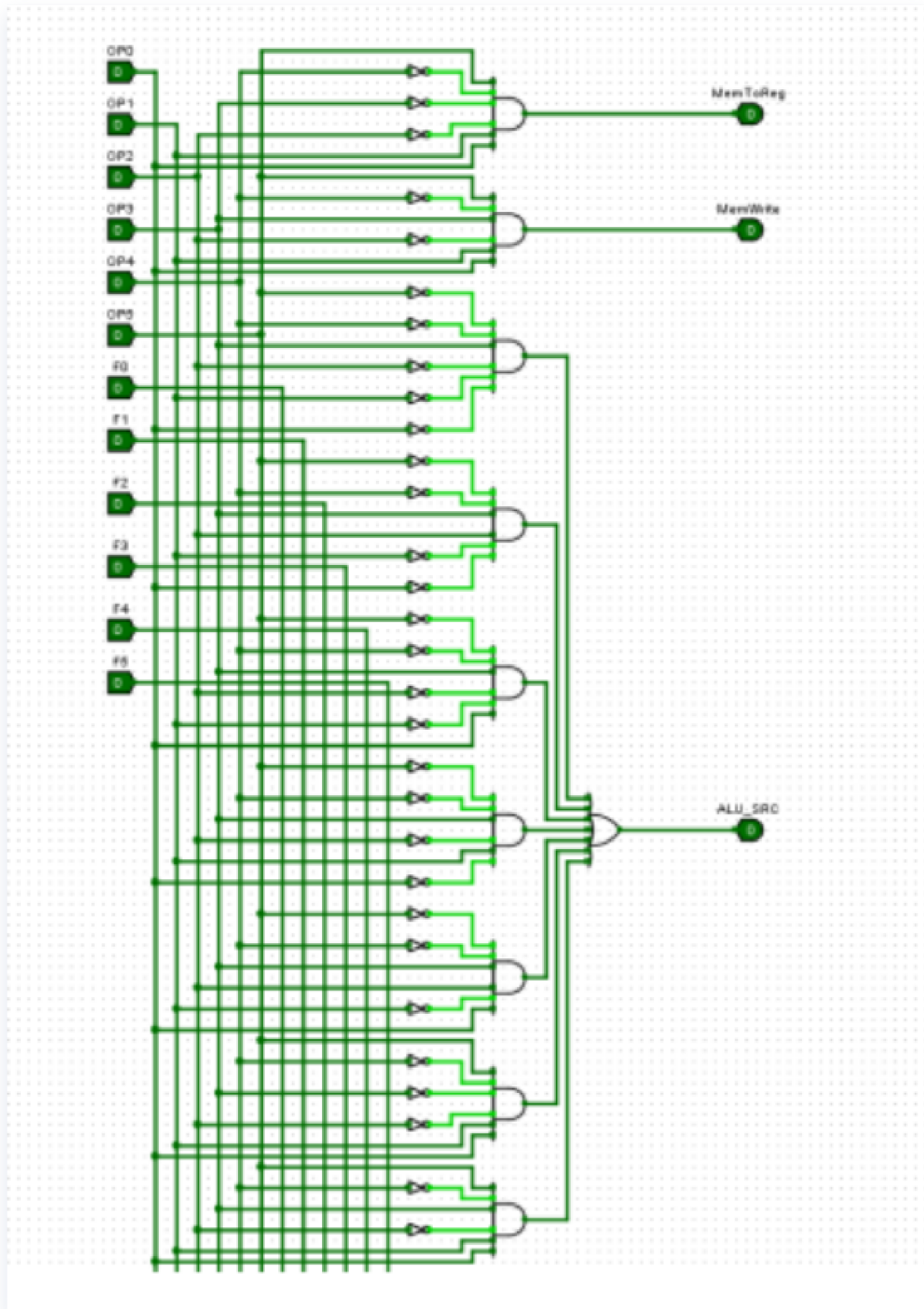


单周期 MIPS 处理器对测试程序的运行结果是正确的。以 `sum_mips` 为例，截图中数据存储器 `000` 单元为 `00000037`，即十进制 55，和 `1+2+...+10` 的结果一致；`fib_mips` 截图中前几项为 `00000000`、`00000001`、`00000001`、`00000002`，符合 Fibonacci 数列；`sort3_mips_bus` 与 `sort4_mips_bus` 的截图中又分别得到 `5、4、3、2、1` 和 `1、2、3、4、5` 的结果。这说明该处理器能够在—个时钟周期内完成每条指令的执行，并正确支持算术、访存、比较、分支和跳转等操作。

(9) 该单周期 MIPS 处理器的控制器能不能采用微程序控制器的方法设计？为什么？

理论上可以借鉴微程序“按步骤产生控制信号”的思想，但不适合本实验中的单周期处理器。因为单周期结构要求一条指令在一个时钟周期内完成取指、译码、执行、访存和写回，而微程序控制通常需要多个微周期逐步完成这些动作。若强行采用典型微程序方式，就会把一条机器指令拆成多步执行，从而破坏单周期处理器的基本定义。因此本实验中的单周期 MIPS 更适合采用硬布线组合逻辑直接生成控制信号。

(10) 分析单周期MIPS处理器控制信号的生成电路原理（输入输出的逻辑，拷屏+文字解释）



该电路的输入主要是指令中的 `opcode`、`funct` 等字段，输出则包括 `RegDst`、`ALUSrc`、`MemToReg`、`RegWrite`、`MemRead`、`MemWrite`、`Branch`、`Jump`、`ALUOp` 等控制信号。控制器先根据 `opcode` 区分指令大类，例如访存类、分支类、R型指令和跳转类，再结合 `funct` 进一步确定 R 型指令对应的具体 ALU 运算。随后这些控制信号分别送往寄存器堆写地址选择器、ALU 输入选择

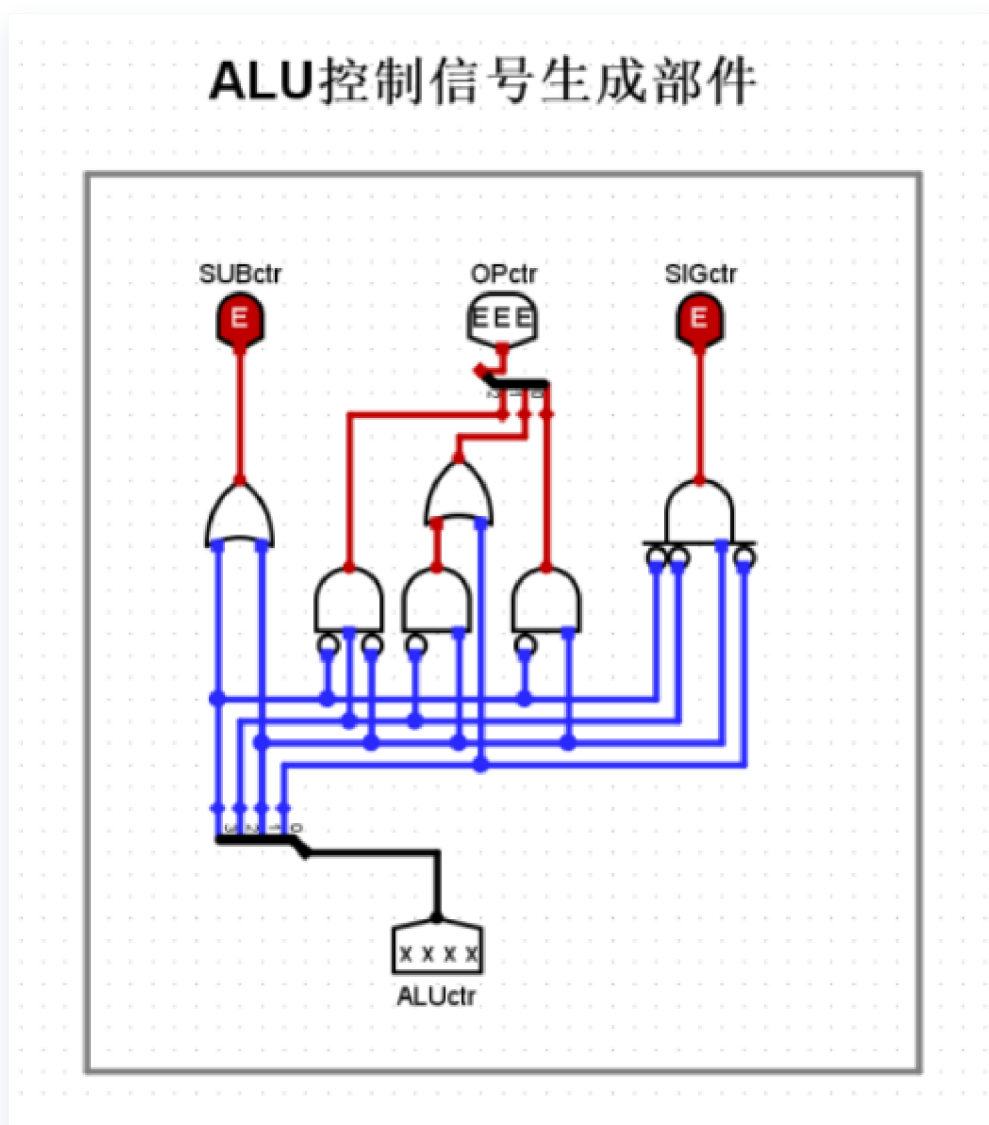
器、数据存储器读写使能以及 PC 更新通路，使数据通路按照当前指令要求工作。也就是说，该模块本质上是“对指令字段做组合译码，再把译码结果分发到各功能部件”。

(11) 单周期 RISC-V 处理器的控制器能不能采用微程序控制器的方法设计？为什么？

同样地，理论上可以用微程序思想描述控制过程，但不适合单周期 RISC-V 处理器。因为本实验中的单周期 RISC-V 也是要求一条指令在一个时钟周期内完成全部操作，而微程序控制通常依赖多个微周期来顺序发出控制信号。若采用传统微程序方式，就会把单周期结构改造成多周期结构，不再符合原有设计目标。因此本实验的 RISC-V 控制器更适合用硬布线逻辑根据 `opcode`、`funct3`、`funct7` 等字段直接生成控制信号。

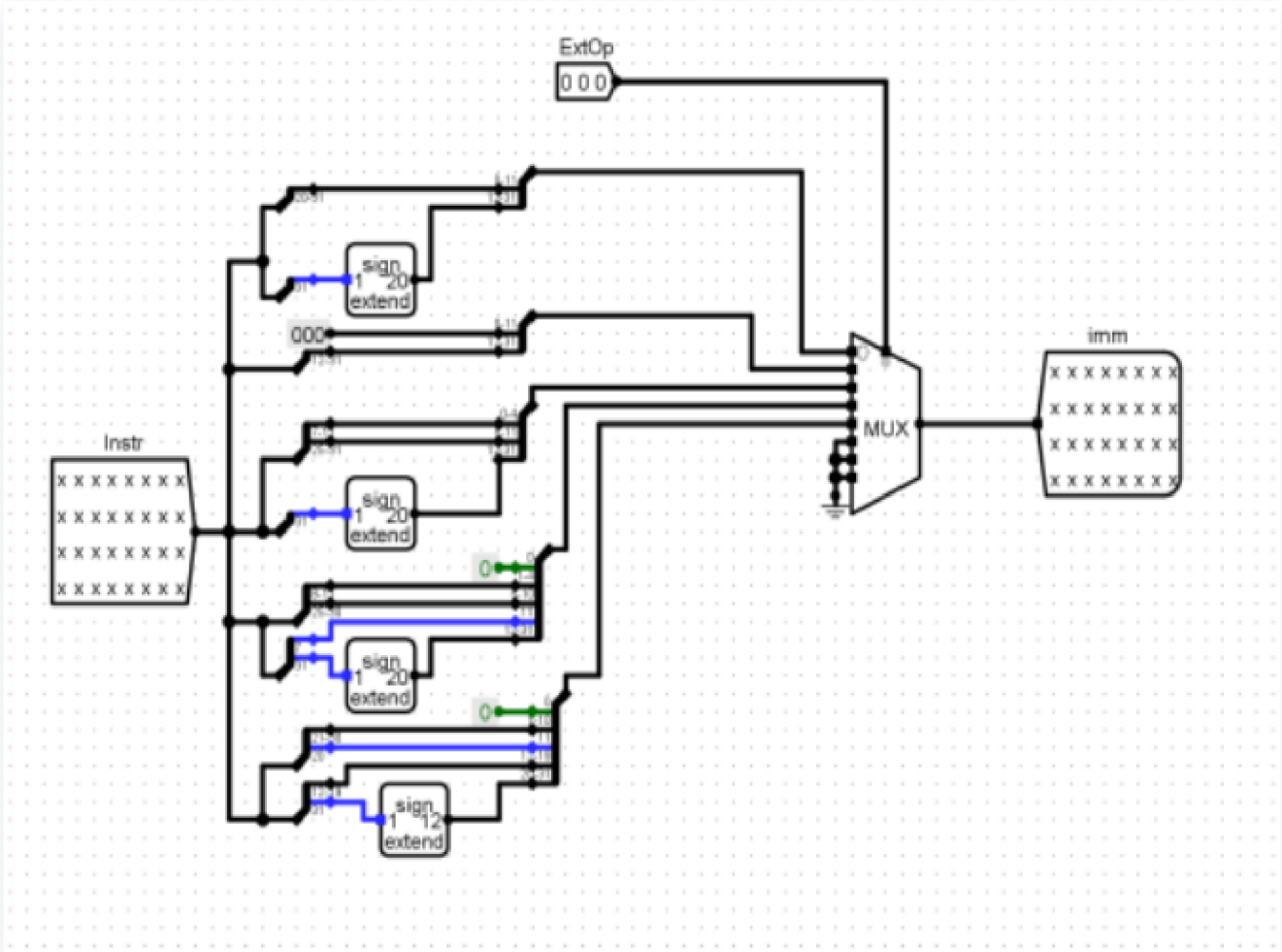
(12) 请分析单周期 RISC-V 处理器（硬布线控制器）的下述电路原理：

a) ALU 控制信号生成部件



b)立即数扩展部件

RISC-V 的 ALU 控制信号生成部件以 `opcode`、`funct3`、`funct7` 等指令字段作为输入，判断当前应执行的是 `add` 指令、`slt`、`sltu`、`ori` 等哪一种运算，再输出对应的 `ALUctr` 去控制 ALU。这样即使不同指令的大类相同，只要功能字段不同，ALU 仍能执行正确的具体操作。立即数扩展部件则根据指令格式对立即数字段进行提取和拼接，例如 I 型、S 型、B 型、J 型指令的立即数字段位置各不相同，扩展部件需要按照格式规则组合并进行符号扩展，最后输出 32 位立即数。该立即数可用于 ALU 运算、访存地址计算以及分支跳转目标地址的形成，因此它是单周期 RISC-V 数据通路中的关键部件。

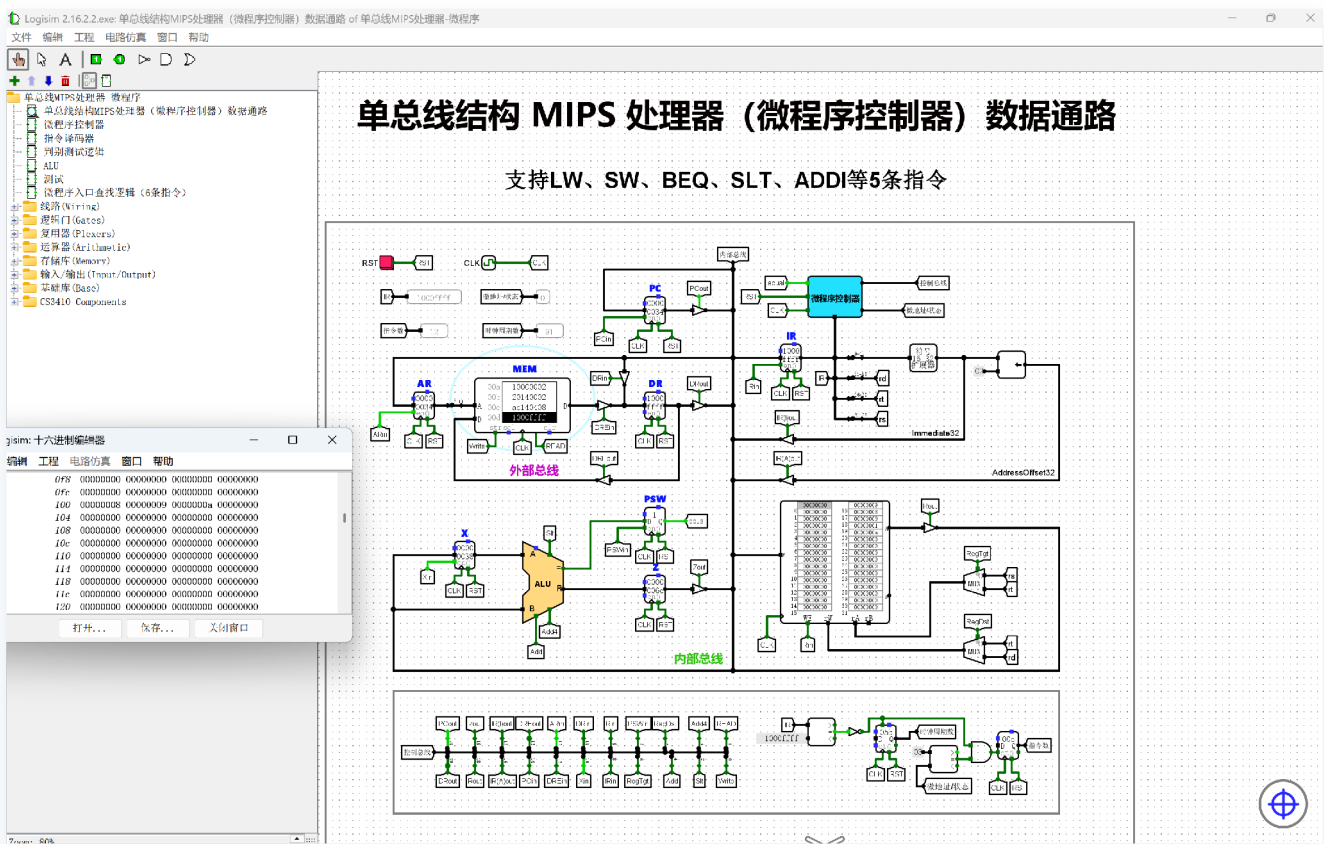


3.2 设计性实验

(1) 单总线结构 MIPS 处理器（微程序控制器）（6条指令）

test2

截图

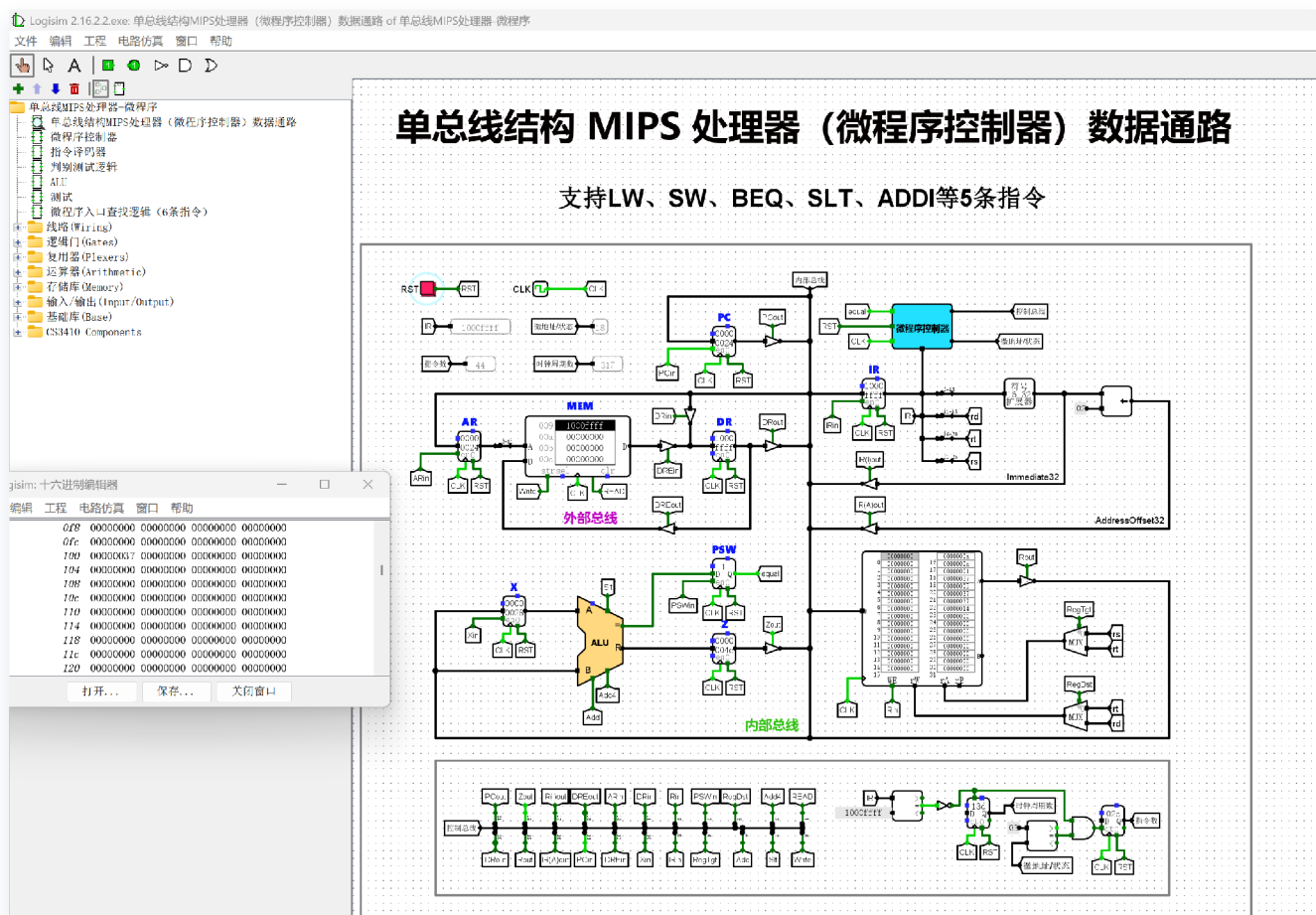


实验分析

运行 test2.hex 后，程序最终进入结束死循环。截图中 RAM 的 100、101、102 单元分别为 00000008、00000009、0000000a，与 test2.asm 的预期结果一致。其中 0000000a 来自 add 指令完成 1+9 的寄存器加法结果，说明本次使用的 6 条指令版本已经能够正确支持 add 指令。微程序控制器和扩展后的数据通路均未出现异常。虽然截图标题中仍显示为 5 条指令版本，但本实验实际运行的是扩展后的 6 条指令版本。

sum_mips_bus

截图

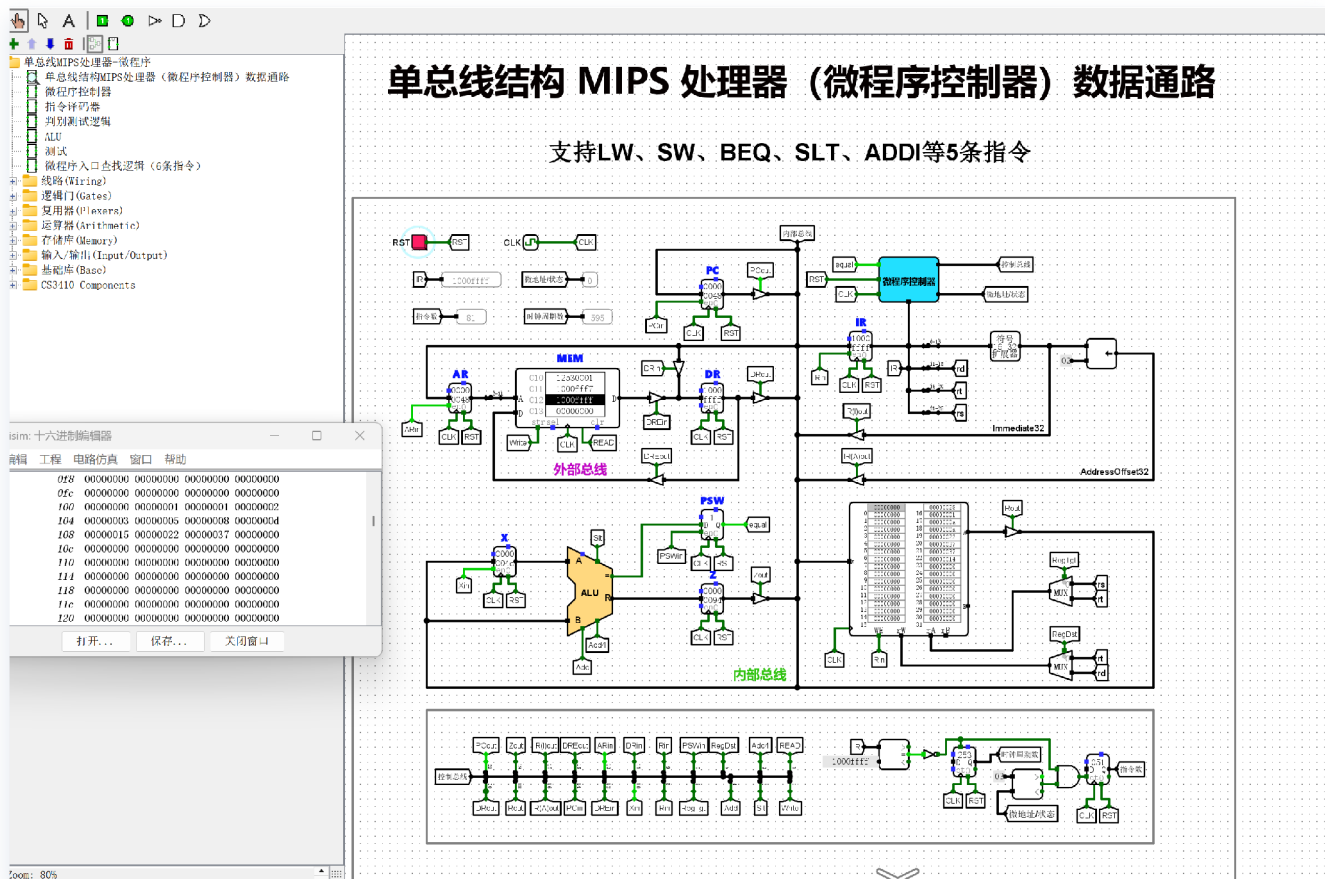


实验分析

运行 `sum_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 100 单元为 00000037，即十进制 55，与 $1+2+...+10$ 的累加和一致。相比原 5 条指令版本，本程序能够成功运行，关键在于扩展后的 add 指令支持了寄存器之间的加法。6 条指令的微程序控制器已经能够正确执行累加循环。

fib_mips_bus

截图

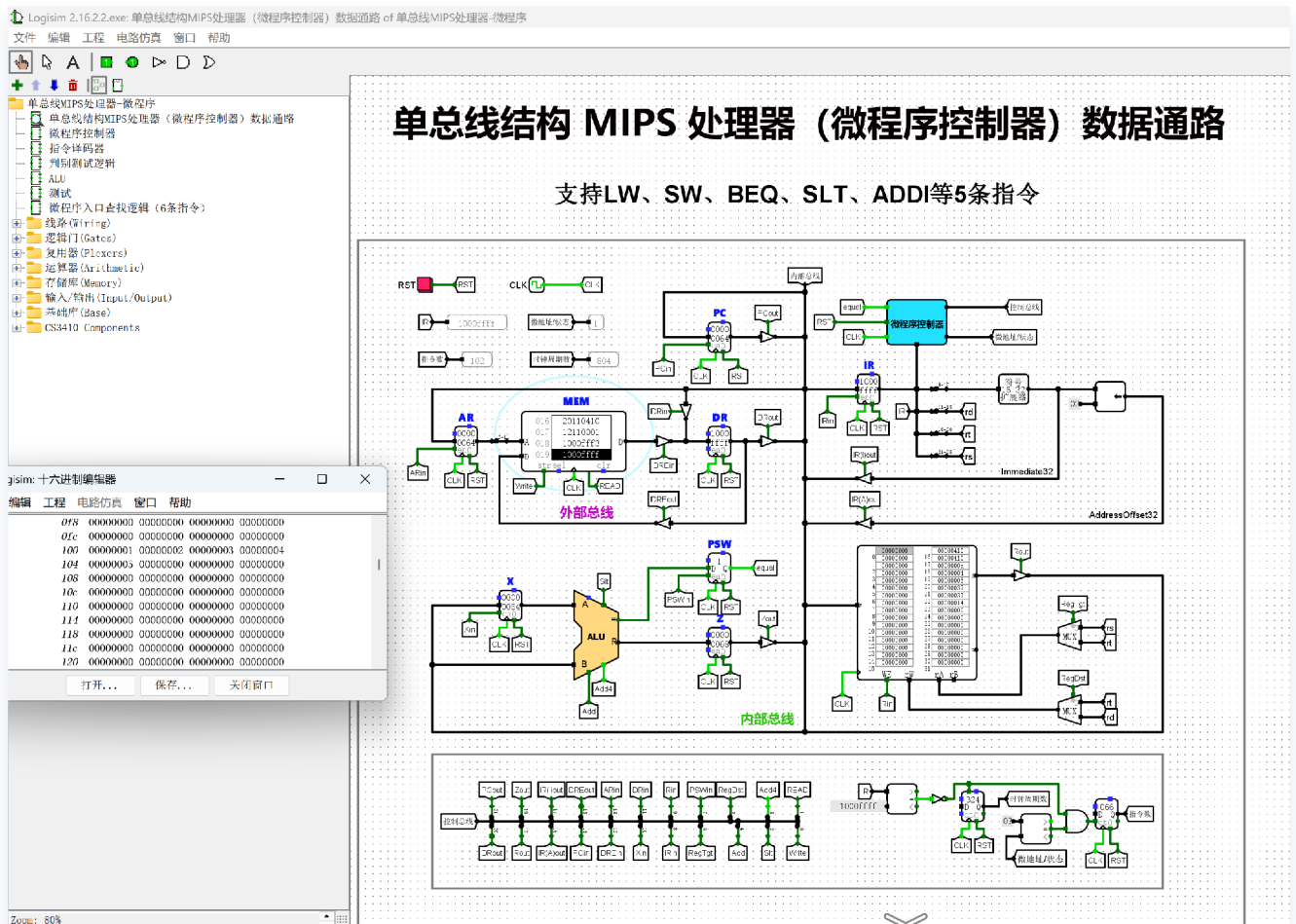


实验分析

运行 `fib_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 从 100 开始依次可以看到 00000000、00000001、00000001、00000002、00000003、00000005、00000008、0000000d、00000015、00000022、00000037，与 Fibonacci 数列前 11 项完全一致。加入 add 指令后，处理器已经能够支持数列递推中大量出现的寄存器加法操作，微程序控制器在循环、写回和地址更新过程中也工作正常。

sort3_mips_bus

截图



实验分析

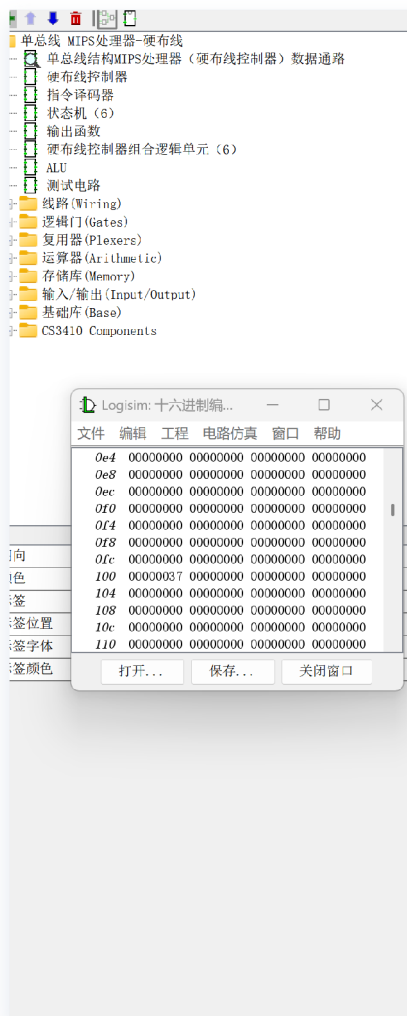
运行 `sort4_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 100~104 单元依次为 00000001、00000002、00000003、00000004、00000005。这说明数据已经按从小到大的顺序完成排序，与预期结果一致。扩展后的 6 条指令微程序版本不仅能支持 add 指令，还能继续正确执行包含多分支、多访存和多次比较的排序程序，运行结果符合预期。

3.3 挑战性实验

(1) 单总线结构 MIPS 处理器（硬布线控制器）（6条指令）

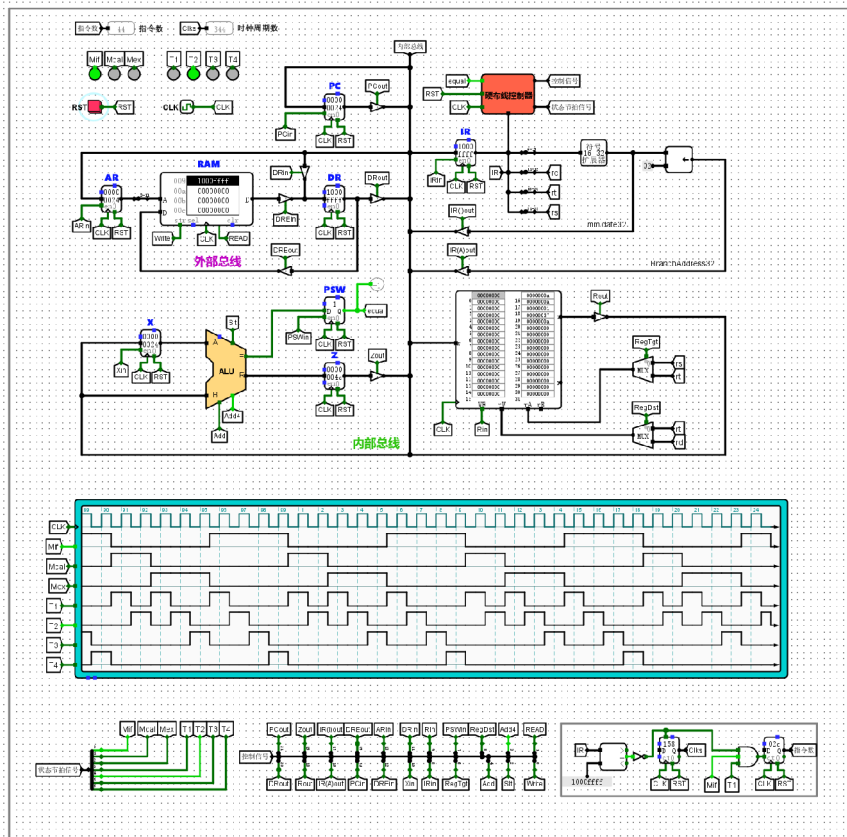
test2

截图



单总线结构 MIPS 处理器（硬布线控制器）数据通路

支持LW、SW、BEQ、SLT、ADDI、ADD等6条指令

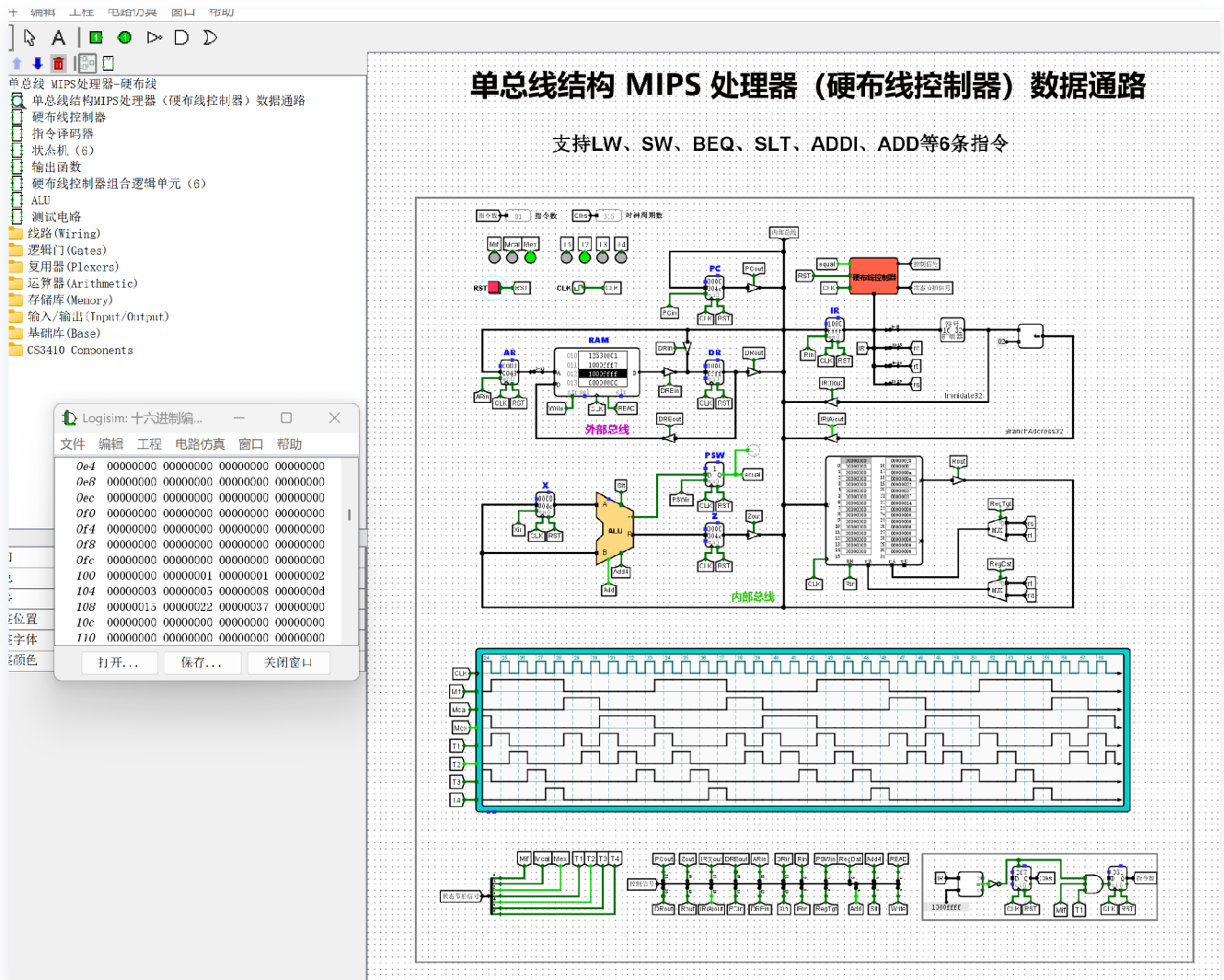


实验分析

运行 `sum_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 100 单元为 00000037，即十进制 55。结果与累加和程序预期一致，说明加入 add 指令后，硬布线控制器版本已经能够完成寄存器间加法和循环累加。程序能够正常运行，也表明扩展后的状态机没有破坏原有 `beq`、`sw` 等指令的时序控制。

fib_mips_bus

截图

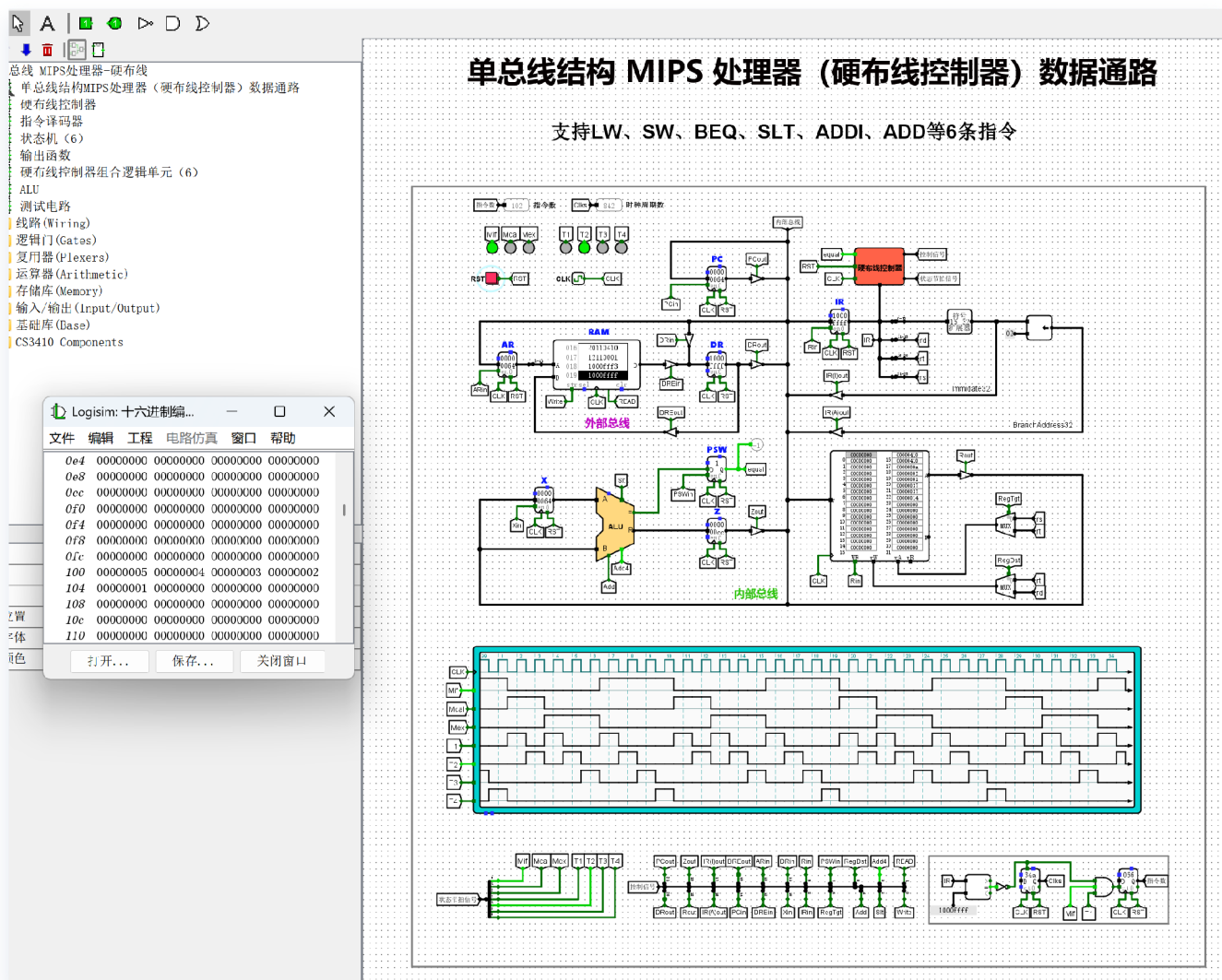


实验分析

运行 `fib_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 从 100 开始依次保存了 00000000、00000001、00000001、00000002、00000003、00000005、00000008、0000000d、00000015、00000022、00000037。这些数据正好对应 Fibonacci 数列前 11 项，说明硬布线控制器已经可以通过状态机和组合逻辑正确完成 add 指令，并支持更复杂的递推程序。访存、分支和寄存器写回都保持正确，本次扩展能够正常运行相关程序。

sort3_mips_bus

截图

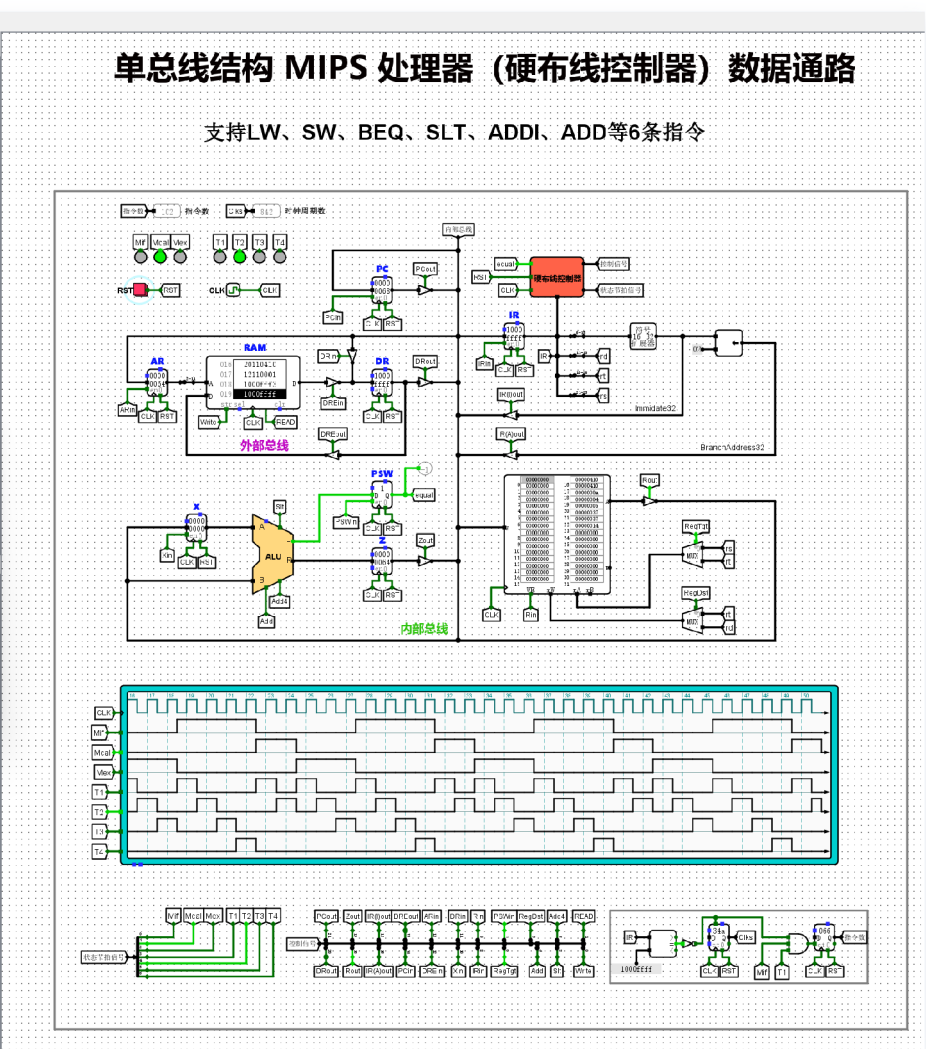
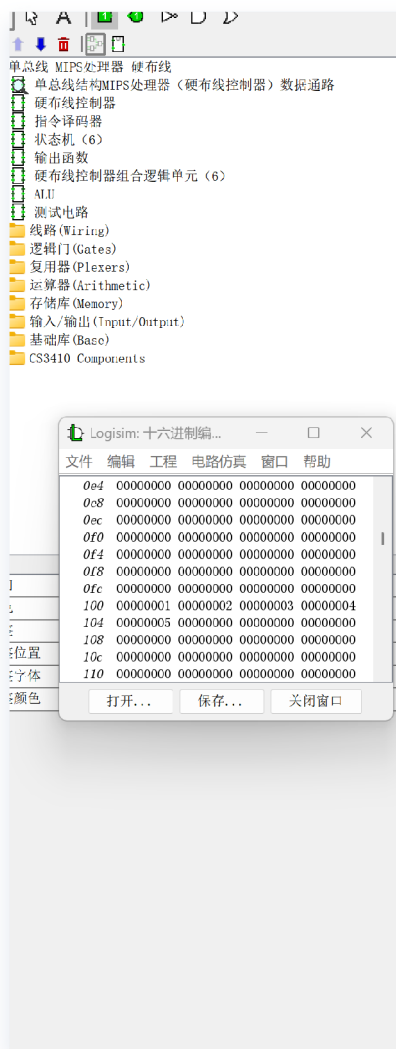


实验分析

运行 `sort3_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 100~104 单元依次为 00000005、00000004、00000003、00000002、00000001。结果说明数据已经按降序正确排好，排序过程中的多次比较、交换和分支都得到了正确执行。虽然排序程序主要依赖基本指令，但扩展后的硬布线控制器在加入 add 指令后仍保持了原有运行效果，未出现异常结果。

sort4_mips_bus

截图



实验分析

运行 `sort4_mips_bus.hex` 后，程序最终停在结束死循环。截图中 RAM 的 100~104 单元依次为 00000001、00000002、00000003、00000004、00000005。升序排序结果与预期结果一致，说明扩展后的硬布线控制器能够完成多分支、多访存和多比较条件下的全部控制。加入 add 指令后，数据通路和控制逻辑仍然协调一致，能够正常运行相关排序程序。